React Component Life Cycle Hands-On Lab — Solution

Building a Blog Application ("blogapp") with componentDidMount() and componentDidCatch()

Objectives

- Explain the need and benefits of the component life cycle
- Identify various life cycle hook methods
- List the sequence of steps in rendering a component

In this hands-on lab, you will learn how to:

- Implement the componentDidMount() hook
- Implement the componentDidCatch() life cycle hook

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Task

Create a React application named "blogapp" that fetches blog posts from a public API (https://jsonplaceholder.typicode.com/posts) using the componentDidMount() life cycle hook, models each post with a Post class, displays the title and body of every post, and reports any errors that occur using the componentDidCatch() life cycle hook.

Solution — Step by Step

Step 1: Create the React project "blogapp"

Open a terminal and run the following command to scaffold the project:

```
npx create-react-app blogapp
```

Step 2: Open the application in VS Code

Navigate into the project folder and open it in Visual Studio Code:

```
cd blogapp
code .
```

Step 3: Create Post.js in the src folder

Add a new file named Post.js under the src folder. This defines a simple Post class used to model each blog post returned by the API:

```
class Post {
  constructor(userId, id, title, body) {
    this.userId = userId;
```

```
    this.id = id;
    this.title = title;
    this.body = body;
  }
}

export default Post;
```

Step 4: Create the Posts class component

Add a new file named Posts.js under the src folder and start the class component:

```
import React, { Component } from 'react';
import Post from './Post';

class Posts extends Component {
  // constructor, loadPosts(), componentDidMount(),
  // componentDidCatch() and render() are added in the
  // following steps
}

export default Posts;
```

Step 5: Initialize state with a list of posts using the constructor

Add a constructor that initializes the component's state with an empty array of posts:

```
constructor(props) {
  super(props);
  this.state = {
    posts: []
  };
}
```

Step 6: Create the loadPosts() method

Add a loadPosts() method that uses the Fetch API to retrieve posts from https://jsonplaceholder.typicode.com/posts, converts each item into a Post object, and assigns the resulting array to the component's state:

```
loadPosts() {
  fetch('https://jsonplaceholder.typicode.com/posts')
    .then(response => response.json())
    .then(data => {
      const posts = data.map(
        item => new Post(item.userId, item.id, item.title, item.body)
      );
      this.setState({ posts: posts });
    })
    .catch(error => {
      throw error;
```

```
    });
}
```

Step 7: Implement componentDidMount()

Add the componentDidMount() life cycle hook so that loadPosts() is called automatically once the component has mounted:

```
componentDidMount() {
  this.loadPosts();
}
```

Step 8: Implement render()

Add the render() method to display the title and body of each post using a heading and a paragraph:

```
render() {
  return (
    <div className="posts">
      <h1>Blog Posts</h1>
      {this.state.posts.map(post => (
        <div key={post.id} className="post">
          <h2>{post.title}</h2>
          <p>{post.body}</p>
        </div>
      ))}
    </div>
  );
}
```

Step 9: Implement componentDidCatch()

Add the componentDidCatch() life cycle hook so that any error occurring inside the component is reported to the user as an alert message:

```
componentDidCatch(error, info) {
  alert('Something went wrong while loading the posts: ' + error.message);
}
```

Complete Posts.js

Putting all the pieces together, the complete Posts.js file looks like this:

```
import React, { Component } from 'react';
import Post from './Post';

class Posts extends Component {
  constructor(props) {
    super(props);
    this.state = {
      posts: []
```

```
    };
  }

  loadPosts() {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(response => response.json())
      .then(data => {
        const posts = data.map(
          item => new Post(item.userId, item.id, item.title, item.body)
        );
        this.setState({ posts: posts });
      })
      .catch(error => {
        throw error;
      });
  }

  componentDidMount() {
    this.loadPosts();
  }

  componentDidCatch(error, info) {
    alert('Something went wrong while loading the posts: ' + error.message);
  }

  render() {
    return (
      <div className="posts">
        <h1>Blog Posts</h1>
        {this.state.posts.map(post => (
          <div key={post.id} className="post">
            <h2>{post.title}</h2>
            <p>{post.body}</p>
          </div>
        ))}
      </div>
    );
  }
}

export default Posts;
```

Step 10: Add the Posts component to App.js

Update App.js to import and render the Posts component:

```
import React from 'react';
import './App.css';
import Posts from './Posts';
```

```
function App() {
  return (
    <div className="App">
      <Posts />
    </div>
  );
}

export default App;
```

Step 11: Build and run the application

In the command prompt, navigate into the blogapp folder (if not already there) and run:

```
cd blogapp
npm start
```

Open a browser window and type the following in the address bar:

```
http://localhost:3000
```

componentDidMount() triggers loadPosts() as soon as the Posts component mounts, fetching the list of posts from the API. render() then displays the title and body of each post on the page. If an error occurs anywhere in the component (for example, a failed network request), componentDidCatch() intercepts it and displays it to the user as an alert.

Result

A React application named "blogapp" was successfully created with a Post class to model each blog post and a Posts class component that fetches data from https://jsonplaceholder.typicode.com/posts inside componentDidMount(), stores it in state, renders the title and body of every post, and uses componentDidCatch() to surface any errors as alert messages. The Posts component was added to App.js and verified by running the application locally at http://localhost:3000.